

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DIPLOMĂ

CultureQuest - Platforma mobilă de explorare urbană bazată pe proximitate, cu recomandări personalizate prin învățare federată

Andrei Cozma

Coordonator științific:

Prof. dr. ing. Radu-Ioan Ciobanu

BUCUREȘTI

2026

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



DIPLOMA PROJECT

CultureQuest - Proximity-Based Mobile Platform for Urban
Exploration with Personalized Recommendations via Federated
Learning

Andrei Cozma

Thesis advisor:

Prof. dr. ing. Radu-Ioan Ciobanu

BUCHAREST

2026

CUPRINS

1	Introducere	1
1.1	Context	1
1.2	Definirea problemei	1
1.3	Obiective	2
1.4	Soluția propusă	2
1.5	Rezultatele obținute	3
1.6	Structura lucrării	3
2	Analiza Cerințelor	4
2.1	Utilizatori țintă și cazuri de utilizare	4
2.2	Cerințe funcționale	4
2.3	Cerințe non-funcționale	5
2.4	Domeniul de aplicare al proiectului	6
3	Soluții Existente	7
3.1	Aplicații existente pentru explorare urbană	7
3.2	Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări	8
3.3	Învățarea federată: Stadiul actual	9
3.4	Alternative respinse	10
3.5	Tehnologiile utilizate și justificarea acestora	11
3.5.1	Flutter și Riverpod	11
3.5.2	FastAPI, MongoDB și Redis	11
3.5.3	Motorul de Rutare OSRM	12
3.5.4	Perceptron multi-strat	12
4	Soluția Propusă	14
4.1	Prezentare generală a sistemului	14

4.2	Arhitectura aplicației mobile	14
4.3	Arhitectura serviciilor <i>backend</i>	14
4.4	Proiectarea sistemului de învățare federată	14
4.4.1	Arhitectura modelului	14
4.4.2	Antrenarea locală	14
4.4.3	Agregare asincronă și reducerea în funcție de vechime	14
4.4.4	Limitarea gradientilor	14
4.4.5	Confidențialitate diferențială	14
4.4.6	Controlul concurenței	14
4.5	Fluxul de generare a rutelor	14
4.6	Proiectarea elementelor de gamificare	14
4.7	Hartă și navigare	14
4.8	Arhitectura de confidențialitate	14
5	Detalii de Implementare	15
5.1	Implementarea aplicației mobile	15
5.1.1	Serviciul client de învățare federată	15
5.1.2	Adăugarea de zgomot pentru asigurarea confidențialității diferențiale	15
5.1.3	Bufferul local de interacțiuni și persistența datelor	15
5.1.4	Stocarea temporară a obiectivelor pentru funcționarea offline	15
5.1.5	Algoritmul de generare a rutelor	15
5.1.6	Sistemul de misiuni și tehnologia de <i>geofencing</i>	15
5.2	Implementarea serviciilor <i>backend</i>	15
5.2.1	Serviciul de agregare	15
5.2.2	Punctele de acces API	15
5.2.3	Sistemul de recenzii și moderarea conținutului	15
5.3	Fluxul etapei de pre-antrenare	15
5.4	<i>Framework-ul</i> pentru simularea procesului de învățare federată	15

5.5	Dificultăți întâmpinate în procesul de implementare	15
6	Evaluare	16
6.1	Corectitudinea funcțională	16
6.2	Performanța modelului de învățare federată	16
6.2.1	Rezultatele etapei de pre-antrenare	16
6.2.2	Rezultatele simulării procesului de învățare federată	16
6.3	Evaluarea confidențialității și robusteții	16
6.4	Analiză comparativă și discuții	16
7	Concluzii	17
7.1	Concluzii generale	17
7.2	Direcții de cercetare și lucrări viitoare	17
	Bibliografie	18
	Anexe	20
	Anexa A Extrase de cod	21

SINOPSIS

CultureQuest este o platformă mobilă de explorare urbană bazată pe proximitate, oferind recomandări personalizate legate de obiectivele turistice și recreative dintr-un oraș, fără ca datele utilizatorului să părăsească dispozitivul. Problema pe care o tratez este că aplicațiile de tip *city-guide* fie oferă aceleași sugestii tuturor, indiferent de interese, fie personalizează recomandările prin salvarea centralizată a istoricului de interacțiuni cu obiectivele sau monitorizarea traseului, ceea ce ridică probleme din punctul de vedere al confidențialității. Pentru a evita acest compromis, am implementat o aplicație Flutter în care datele sunt salvate doar local, interacțiunile vizibile altor utilizatori precum recenziile sunt anonime și adăugate strict de vizitatori, iar rutele turistice generate sunt individualizate prin intermediul unei rețele neurale de tip perceptron multi-strat (MLP, *Multi-Layer Perceptron*). Modelul global inițial este pre-antrenat pe seturi de date publice, iar acesta este modificat și îmbunătățit prin tehnici de învățare federată (FL, *Federated Learning*) rulate asincron pe dispozitivele clienților. Recomandările de rute combină scorul oferit de model pentru o atracție turistică cu proximitatea acesteia, iar un sistem de misiuni gamificate încurajează explorarea și colectarea de noi date. Printr-o simulare care include mai multe acțiuni - mai exact 600 - se confirmă funcționarea fluxului FL în mod corespunzător.

ABSTRACT

CultureQuest is a proximity-based mobile urban exploration platform, offering personalized recommendations for tourist and recreational attractions in a city, without the user's data leaving the device. The problem I address is that city-guide type applications either offer the same suggestions to all users, regardless of interests, or personalize recommendations by centrally storing a user's interaction history with landmarks or by tracking their route, both of which raise privacy concerns. To avoid this trade-off, I developed a Flutter application where data is saved only locally, interactions visible to other users such as reviews are anonymous and can only be added by actual visitors and the generated tourist routes are personalized through a multi-layer perceptron (MLP) neural network. The initial global model is pre-trained on public datasets, and it is modified and improved through federated learning (FL) techniques run asynchronously on the client devices. Route recommendations combine the model's score for a tourist attraction with its proximity, while a gamified quest system encourages exploration and data collection. A simulation that includes multiple client actions - 600, to be exact - confirms that the FL flow is working properly.

1 INTRODUCERE

Acest capitol prezintă contextul în care se situează proiectul și problema care îl motivează (1.1, 1.2), obiectivele propuse (1.3), o privire de ansamblu asupra soluției dezvoltate (1.4) și a rezultatelor obținute (1.5), precum și modul în care este organizată restul lucrării (1.6).

1.1 Context

Telefoanele mobile au devenit, în ultimii ani, principalul instrument prin care utilizatorii interacționează cu mediul din jurul lor, iar aplicațiile mobile bazate pe locație - de la Google Maps, TripAdvisor și GetYourGuide, la diverse aplicații de tip *city-guide* - au devenit principala modalitate prin care turiștii și localnicii descoperă obiective turistice și culturale dintr-un oraș și primesc recomandări din ce în ce mai adaptate propriilor interese [5].

Pentru a oferi recomandări personalizate, aceste aplicații se bazează pe colectarea și procesarea centralizată a istoricului de interacțiuni al utilizatorului - locurile vizitate, recenziile adăugate, timpul petrecut în diverse locații - lucru care ridică probleme de confidențialitate, întrucât utilizatorul pierde controlul asupra modului în care îi sunt folosite și stocate datele legate de comportament.

Învățarea federată (FL, *Federated Learning*) apare ca alternativă la acest model centralizat: modelul este antrenat local, pe dispozitivul utilizatorului, folosind datele acestuia, iar către server sunt trimise doar actualizările rezultate, nu datele brute, pentru a fi agregate într-un model global. FL este deja folosită la scară destul de mare în sisteme de producție [4], ceea ce motivează aplicarea ei și în cadrul acestei platforme, pentru a oferi recomandări de rute turistice fără a compromite confidențialitatea clienților.

1.2 Definirea problemei

Majoritatea aplicațiilor de explorare urbană dezvoltate oferă, în absența unui profil construit din acțiunile turiștilor pe platformă, aceleași rute tuturor, indiferent de interesele fiecăruia, chiar dacă acestea sunt legate de artă, gastronomie, natură sau alte evenimente culturale. Această abordare generică este mai simplă de dezvoltat și de întreținut, însă nu ține cont de faptul că două persoane care vizitează același oraș pot avea preferințe complet diferite în ceea ce privește modul în care doresc să îl exploreze.

Varianta prin care o aplicație devine personalizată presupune, de regulă, ca istoricul de interacțiuni al utilizatorului să fie încărcat și procesat pe un server central, exact tipul de practică care aduce în discuție problemele de confidențialitate menționate mai sus. Utiliza-

torului i se prezintă, astfel, un compromis: fie acceptă recomandări generice, fie acceptă ca datele sale comportamentale să fie colectate central pentru a primi recomandări adaptate intereselor sale - iar acest compromis este punctul de plecare al proiectului de față.

1.3 Obiective

Pornind de la acest compromis, proiectul de față își propune să arate că personalizarea și păstrarea confidențialității nu se exclud reciproc, prin trei obiective principale:

- (a) O aplicație mobilă prin care utilizatorul poate descoperi obiective culturale dintr-un oraș printr-o explorare bazată pe proximitate, cu un sistem de *quest-uri* și puncte obținute pentru interacțiunile pe platformă, care îl încurajează să viziteze locuri noi.
- (b) Personalizarea recomandărilor de rute pentru fiecare utilizator, fără ca datele acestuia - istoricul de interacțiuni, locațiile vizitate - să părăsească dispozitivul.
- (c) Implementarea unui flux complet de FL: pre-antrenarea modelului global, ajustarea fină (*fine-tuning*), antrenarea locală prin FedAvg [11] și agregarea asincronă a actualizărilor primite de la clienți, cu garanții de confidențialitate.

1.4 Soluția propusă

Soluția dezvoltată este o aplicație mobilă realizată în Flutter, prin care utilizatorul poate vedea pe hartă obiectivele turistice și evenimentele din apropiere. Pe lângă explorare, aplicația oferă un sistem de *quest-uri*, care încurajează vizitarea unor obiective noi, dar și posibilitatea de a lăsa recenzii, povești sau chiar alte *quest-uri* pentru locurile vizitate - tot ce adaugă utilizatorii devine, în timp, parte din imaginea pe care comunitatea o are despre fiecare atracție.

Pentru personalizare, fiecare utilizator are pe dispozitiv un model mic, un perceptron multi-strat (MLP, *Multi-Layer Perceptron*) cu un strat de intrare de 22 de caracteristici, două straturi ascunse (32 și 16 neuroni) și un neuron de ieșire, care reprezintă un scor de recomandare pentru fiecare obiectiv. Pe baza acestui scor, fiecărui obiectiv i se asociază o etichetă care arată cât de recomandat este pentru utilizator, în funcție de interesele acestuia, de ora curentă, de tipul locului etc., iar la generarea rutei, scorul fiecărui obiectiv este combinat cu distanța față de poziția curentă. Modelul global inițial este pre-antrenat cu ajutorul unor seturi de date din mediul online, apoi antrenat pe dispozitiv prin FL, iar *backend-ul* (FastAPI, MongoDB, Redis) coordonează învățarea asincronă.

1.5 Rezultatele obținute

La nivel general, rezultatul este o aplicație Flutter funcțională, care reunește harta cu obiective și evenimente din apropiere, generarea de rute personalizate și sistemul de *quest-uri*. Personalizarea descrisă în secțiunea anterioară este vizibilă direct în aplicație, pentru fiecare obiectiv din apropiere și în cadrul rutelor generate.

Pe partea de FL, modelul de pe dispozitiv pornește de la o variantă a modelului global trecută și prin etapa de *fine-tuning*, iar antrenarea locală se face prin FedAvg. Agregarea pe server este asincronă, cu o reducere în funcție de vechimea (*staleness discount*) actualizărilor venite de la clienți [12] - o metodă prin care influența lor asupra modelului global este diminuată. O simulare de 600 de runde confirmă funcționarea în parametri buni a fluxului FL.

1.6 Structura lucrării

Capitolul 1 a introdus, la nivel general, contextul, problema, obiectivele, soluția propusă și rezultatele obținute. Capitolele următoare intră în detaliu pe fiecare dintre aceste aspecte.

Capitolul 2 stabilește cerințele aplicației, pornind de la cazurile de utilizare ale unui turist sau localnic care explorează un oraș, și le împarte în cerințe funcționale și non-funcționale. Tot aici se delimitează și domeniul de aplicare al proiectului.

Capitolul 3 trece în revistă aplicațiile de explorare urbană existente și modul în care acestea personalizează recomandările, comparând abordările centralizate cu FL. Se poziționează CultureQuest pe baza taxonomiei FL, se explică de ce alte alternative de algoritmi FL nu se potrivesc design-ului asincron al platformei și se justifică tehnologiile alese.

Capitolul 4 detaliază arhitectura propusă a sistemului. Aici se discută structura aplicației mobile și a serviciilor *backend*, proiectarea modelului MLP și a procesului FL, fluxul de generare a rutelor, elementele de gamificare și arhitectura de confidențialitate.

După proiectare, **Capitolul 5** trece la implementarea efectivă - serviciul client FL și mecanismul de confidențialitate diferențială (DP, *Differential Privacy*), serviciile *backend* de agregare și de moderare a conținutului, etapa de pre-antrenare și *framework-ul* folosit pentru simularea procesului FL.

Capitolul 6 evaluează rezultatele obținute - corectitudinea funcțională a aplicației, performanța modelului rezultată din pre-antrenare și din fluxul FL, efectul DP și al limitării asupra robusteții - și include o discuție comparativă față de alternativele respinse.

Capitolul 7 încheie lucrarea cu o privire de ansamblu asupra obiectivelor stabilite inițial, în raport cu ce a fost dezvoltat și propune direcții viitoare - de exemplu, personalizarea suplimentară per utilizator sau alte tipuri de agregare în ceea ce privește modelul rețelei neurale.

2 ANALIZA CERINȚELOR

Capitolul de față detaliază cerințele care reies din obiectivele stabilite la 1.3: cine sunt utilizatorii și ce cazuri de utilizare trebuie acoperite (2.1), ce funcționalități rezultă de aici (2.2), ce constrângeri non-funcționale se impun (2.3) și ce rămâne în afara domeniului de aplicare (2.4). Toate aceste cerințe stau la baza deciziilor de proiectare din Capitolul 4, fiind verificate ulterior în evaluarea din Capitolul 6.

2.1 Utilizatori țintă și cazuri de utilizare

Utilizatorul principal al aplicației este un turist sau un localnic care explorează un oraș - sau o zonă a acestuia - cu dorința de a vizita obiective turistice sau culturale din proximitate, adaptate intereselor proprii. Primul caz de utilizare este cel de generare de rute și suport pe parcursul urmăririi acestora, în care utilizatorul vede pe hartă poziția sa curentă, locurile de interes din apropiere și primește o rută care combină preferințele sale cu acestea. Mai mult, interacțiunile cu aplicația - vizite, misiuni (*quest-uri*) finalizate, recenzii - determină ajustarea, prin FL, a modelului de pe dispozitiv - și, implicit, și a celui global - fără ca istoricul brut al interacțiunilor să ajungă pe server. Un al doilea caz de utilizare este cel de contribuție de conținut, în care utilizatorul poate propune obiective noi, povești, misiuni sau recenzii, toate acestea necesitând aprobarea de către alte persoane care folosesc aplicația sau de către un administrator. Prin conturile de tip administrator, relevante mai ales pentru 2.2, se aprobă sau se respinge conținutul propus de utilizatorii obișnuiți.

2.2 Cerințe funcționale

Pornind de la cazurile de utilizare din secțiunea anterioară, cerințele funcționale au fost sintetizate în tabelul 1, fiecare cu o descriere și secțiunile din Capitolele 4-6 unde cerința este tratată. Primul grup (de la **FR-1** la **FR-4**) sumarizează bucla principală de explorare: afișarea pe hartă a obiectivelor turistice din proximitate, generarea rutei care combină scorul modelului de tip MLP cu distanța față de poziția curentă, *quest-urile* disponibile la obiectivele vizitate și posibilitatea de a adăuga sau vizualiza recenzii.

Al doilea grup (de la **FR-5** la **FR-7**) ține de personalizare și administrare: **FR-5** descrie fluxul procesului FL - primirea parametrilor modelului actualizat de la server, urmată de antrenarea locală, fără ca istoricul interacțiunilor să ajungă pe server, **FR-6** descrie controalele de confidențialitate - asigurarea DP, vizibilitatea asupra datelor, iar **FR-7** descrie rolul de administrator - aprobarea sau respingerea conținutului propus.

Tabelul 1: Cerințe funcționale

ID	Descriere	Secțiune
FR-1	Obiective din proximitate pe hartă.	4.7, 5.1.4, 6.1
FR-2	Rute personalizate: scor model MLP și proximitate.	4.5, 5.1.5, 6.1
FR-3	<i>Quest-uri</i> la obiectivele vizitate; puncte și progres.	4.6, 5.1.6, 6.1
FR-4	Adăugare și vizualizare recenzii pentru obiective.	5.2.3, 6.1
FR-5	Sincronizare model și antrenare locală, fără date colectate pe server.	4.4, 5.1.1–5.1.3, 5.2.1, 6.2
FR-6	Zgomot DP aplicat pe actualizări; vizibilitate asupra datelor trimise.	4.8, 5.1.2, 6.3
FR-7	Administrator: aprobă sau respinge conținut propus (obiective, povești, <i>quest-uri</i> , recenzii).	5.2.3, 6.1

2.3 Cerințe non-funcționale

La fel ca în secțiunea 2.2, cerințele non-funcționale au fost sintetizate în tabelul 2, cu aceeași structură (descriere și secțiunile din Capitolele 4-6 unde sunt tratate). Primul grup (de la **NFR-1** la **NFR-3**) are legătură cu constrângerile impuse pe dispozitiv: **NFR-1** cere ca nicio informație privată despre utilizator - exceptând lista de interese - să nu părăsească telefonul, **NFR-2** cere ca obiectivele turistice să rămână disponibile și fără conexiune la internet, iar **NFR-3** cere ca antrenarea locală să nu fie complexă din punct de vedere computațional pentru un telefon obișnuit - ceea ce a motivat alegerea unui model MLP de dimensiuni reduse.

Al doilea grup ține de partea de *backend*: **NFR-4** impune ca serviciul de agregare să suporte actualizări asincrone primite de la mai mulți clienți - fără conflicte între runde, iar **NFR-5** impune ca utilizatorii să primească recomandări utile încă de la început printr-un model global pre-antrenat [4] - și din caracteristici bazate pe ariile de interes asociate fiecărui obiectiv - până când procesul FL își face efectul.

Tabelul 2: Cerințe non-funcționale

ID	Descriere	Secțiune
NFR-1	Confidențialitate: nicio interacțiune a utilizatorului nu părăsește dispozitivul.	4.8, 5.1.2–5.1.3, 6.3
NFR-2	Reziliență offline: obiective turistice stocate în <i>cache</i> local.	5.1.4, 6.1
NFR-3	Cost redus de antrenare pe dispozitiv.	4.4.1, 5.1.1, 6.2.1
NFR-4	Scalabilitatea <i>backend-ului</i> la actualizările asincrone.	4.4.6, 5.2.1, 6.2.2
NFR-5	<i>Cold start</i> : recomandări utile pentru utilizatori folosind modelul pre-antrenat.	4.4.1, 5.3, 6.2.1

2.4 Domeniul de aplicare al proiectului

Cerințele funcționale și non-funcționale stabilite anterior delimitează domeniul de aplicare al proiectului: aplicația mobilă cu recomandări de rute turistice, pornind de la pre-antrenarea globală și fiind îmbunătățite prin FL. Rămâne în afara domeniului de aplicare personalizarea suplimentară la nivel de utilizator prin straturi separate ale modelului MLP (*personal head*), care nu ar fi trimise către server, rezultând într-o arhitectură de tip FedPer [3] - propusă ca direcție de lucrări viitoare, dar neimplementată în versiunea curentă.

Având stabilite cerințele și limitele proiectului, Capitolul 3 trece în revistă soluțiile existente pentru explorare urbană (Google Maps, TripAdvisor, alte aplicații de tip *city-guide*) și realizează o analiză comparativă între abordările centralizate și cele bazate pe FL pentru recomandări.

3 SOLUȚII EXISTENTE

Înainte de proiectarea propriu-zisă a soluției, acest capitol analizează contextul în care se înscrie CultureQuest. Aplicațiile de explorare urbană existente oferă un grad bun de personalizare a recomandărilor, însă acestea se bazează pe sisteme centralizate de colectare a datelor (3.1), de unde și comparația cu abordările bazate pe FL (3.2) și cu stadiul actual al cercetării din domeniu, folosit pentru a poziționa CultureQuest în cadrul soluțiilor FL (3.3). Pe baza acestei analize, capitolul explică de ce o parte dintre cele mai cunoscute variante de algoritmi FL nu se potrivesc design-ului asincron al platformei (3.4) și justifică restul tehnologiilor folosite în implementare (3.5).

3.1 Aplicații existente pentru explorare urbană

Platformele mobile de recomandare pentru turiști [5] oferă un anumit nivel de personalizare bazat pe istoricul de activitate al utilizatorului, stocat și procesat pe serverele furnizorului. Google Maps generează, de exemplu, recomandări pe baza istoricului de locații, a căutărilor și a evaluărilor asociate contului Google¹. TripAdvisor merge pe o logică asemănătoare, prin faptul că sugestiile de obiective și restaurante sunt adaptate pe baza recenziilor citite, a căutărilor anterioare și a locațiilor vizitate, tot prin contul de utilizator, procesat central². În ambele cazuri, personalizarea funcționează doar pentru că acest istoric de interacțiuni ajunge să fie analizat pe server, nu pe dispozitivul utilizatorului.

Pe lângă aceste platforme axate pe navigare, există și aplicații specializate în activități turistice legate de rezervări, precum GetYourGuide, care recomandă tururi cu ghizi, bilete și experiențe pe baza destinației căutate și a istoricului de căutări sau rezervări al utilizatorului, tot stocat și procesat central³. Spre deosebire de Google Maps sau TripAdvisor, accentul este pus pe activități plătite și predefinite, nu pe explorarea liberă a orașului, așa că nu există rute generate dinamic în funcție de poziția utilizatorului. Dincolo de aceste diferențe, toate trei se bazează pe același principiu - ML centralizat, antrenat pe date de la un număr mare de utilizatori.

La polul opus se situează aplicațiile axate pe confidențialitate, precum Organic Maps, un proiect *open-source* bazat pe date de la OpenStreetMap, fără cont de utilizator și fără colectare de date⁴. Aici personalizarea nu există, iar toți utilizatorii dintr-o zonă văd același

¹Google, pagina oficială despre recomandările personalizate. <https://support.google.com/websearch/answer/17025248?hl=en> (accesat 19.06.2026).

²TriAdvisor, pagina oficială despre funcționarea recomandărilor/politica de confidențialitate. <https://tripadvisor.mediaroom.com/US-privacy-policy> (accesat 19.06.2026).

³GetYourGuide, pagina oficială despre cum funcționează recomandările și politica de confidențialitate. <https://www.getyourguide.com/c/privacy-policy> (accesat 19.06.2026).

⁴Organic Maps, pagina oficială - „no tracking, no data collection”. <https://organicmaps.app/privacy/> (accesat 19.06.2026).

set de obiective, indiferent de interese, pentru că nu există niciun istoric din care acestea să poată fi accesate. Între cele două extreme - fie personalizare cu date colectate central, fie confidențialitate fără personalizare - se situează CultureQuest, unde scorul de personalizare este calculat prin FL, fără ca datele brute să părăsească dispozitivul. Toate aceste cazuri sunt adunate în tabelul 3.

Tabelul 3: Comparatie între aplicații existente de explorare urbană și CultureQuest

Aplicație	Personalizare	Colectare date	ML local	Gamificare
Google Maps	Da	Centralizat	Nu	Nu
TripAdvisor	Da	Centralizat	Nu	Limitată
GetYourGuide	Da	Centralizat	Nu	Nu
Organic Maps	Nu	Fără	Nu	Nu
CultureQuest	Da	Local	Da	Da

3.2 Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări

Sistemele de recomandare clasice se bazează, în general, pe filtrarea colaborativă, care recomandă unui utilizator ceea ce au apreciat alți utilizatori cu interese similare, și pe filtrarea bazată pe conținut, care recomandă obiective similare cu cele apreciate deja de același utilizator [2]. În ambele cazuri, determinarea similarităților are nevoie de o bază de date centralizată cu istoricul de activitate al clienților - exact modelul din spatele recomandărilor descrise în secțiunea anterioară pentru aplicațiile menționate. Avantajul e accesul la un volum mare de date eterogene, din care modelul învață tendințe generale utile chiar și pentru utilizatori noi, pe baza unor profiluri asemănătoare existente în sistem. Prețul acestui avantaj este însă tot cel discutat mai sus. Pentru ca modelul să poată face aceste calcule, istoricul de activitate al fiecărui utilizator trebuie să ajungă și să rămână pe server, sub controlul furnizorului.

Învățarea federată (FL, *Federated Learning*) schimbă direcția, astfel încât modelul ajunge la date, nu invers - antrenarea are loc local, pe dispozitivul fiecărui utilizator, iar pe server ajung doar actualizările rezultate ale modelului, nu istoricul de activitate în sine. Datele rămân astfel pe dispozitiv, iar confidențialitatea nu mai depinde de politicile furnizorului, ci de proiectarea protocolului de agregare. Diferența se vede cel mai clar la pornirea la rece (*cold-start*). O aplicație aflată la început nu are încă un model antrenat suficient din cauza unui număr mic de persoane care o folosesc, așa că recomandările vin dintr-un model global pre-antrenat și din caracteristicile bazate pe conținut ale obiectivelor, la fel ca în cazul centralizat - personalizarea prin FL se construiește abia pe măsură ce utilizatorii interacționează cu aplicația.

3.3 Învățarea federată: Stadiul actual

Învățarea federată a apărut ca direcție de cercetare odată cu FedAvg [11], ca răspuns direct la problema discutată în secțiunea anterioară - cum poate fi antrenat un model pe datele mai multor clienți, fără ca acestea să fie centralizate pe un server. De atunci, domeniul s-a diversificat, acoperind variante de agregare, metode DP, robustețe la clienți adversariali și scenarii de personalizare [6]. Pentru a situa CultureQuest în acest cadru, ne putem ghida după două axe de clasificare devenite standard în literatură: tipul de FL, în funcție de cum sunt distribuite datele între clienți, și modul de desfășurare, în funcție de cine sunt clienții și care este comportamentul lor.

Prima axă, propusă de Yang et al. [13], împarte FL în trei categorii:

- **Orizontal (HFL)** - clienții au același spațiu de caracteristici, dar mulțimi diferite de exemple, fiind situația tipică atunci când același tip de date este colectat de la utilizatori diferiți.
- **Vertical (VFL)** - clienții au în comun o parte din exemple, dar caracteristici diferite, ca în cazul unor organizații care dețin date diferite despre aceiași utilizatori.
- **FL cu transfer (FTL)** - pentru situațiile în care nici exemplele, nici caracteristicile nu se suprapun semnificativ.

CultureQuest se încadrează clar în categoria HFL. Fiecare telefon antrenează același model, pe același spațiu de 22 de caracteristici asociate obiectivelor turistice, diferența dintre clienți fiind doar istoricul propriu de interacțiuni.

A doua axă împarte FL în două moduri de desfășurare [6, 8]:

- **Cross-device** - clienții sunt un număr mare de dispozitive individuale, cu conexiuni nesigure și resurse limitate, fiecare contribuind cu o cantitate mică de date.
- **Cross-silo** - clienții sunt un număr mic de organizații sau servere, cu conexiuni stabile și seturi de date mari.

CultureQuest este, din această perspectivă, un sistem *cross-device*. Telefoanele utilizatorilor sunt conectate la internet intermitent, fiecare cu un volum mic de interacțiuni locale. Un sistem *cross-device* are nevoie de agregare asincronă [12], pentru că acești clienți nu pot fi sincronizați la fiecare rundă de învățare, și de mecanisme de robustețe la actualizări extreme [10]. Figura 1 situează CultureQuest pe ambele axe.

	HFL	VFL	FTL
Cross-Device	CultureQuest		
Cross-Silo			

Figura 1: Poziționarea CultureQuest în taxonomia FL, pe axa tipului de FL (HFL/VFL/FTL) și axa modului de desfășurare (Cross-Device/Cross-Silo). Adaptat din Yang et al. 2019 [13] și Kairouz et al. 2021 [6].

3.4 Alternative respinse

Eterogenitatea datelor între clienți este una dintre provocările cele mai notabile ale FL [8], iar o direcție importantă de cercetare propune algoritmi care corectează deriva (*drift*) dintre actualizările locale ale clienților și modelul global. FedProx [9] adaugă funcției de cost locale un termen proximal, care penalizează distanța dintre ponderile actualizate și cele ale modelului global de la începutul runde de învățare. SCAFFOLD [7] introduce, pentru fiecare client, o variabilă de control care estimează cât de mult diferă actualizarea sa de media cohorței și corectează gradientul local pe baza acestei estimări. FedDyn [1] ajunge la un rezultat similar printr-un regularizator dinamic, recalculat la fiecare rundă pe baza istoricului actualizărilor clientului. Toate cele trei leagă actualizarea unui client de comportamentul celorlalți clienți selectați în aceeași rundă.

Această legătură nu are însă un corespondent în design-ul CultureQuest, unde fiecare rundă agregă modelul global cu actualizarea unui singur client. Nu există o cohortă de la care un client să se abată, nici actualizări paralele de reconciliat la final de rundă - condiții pe care se bazează atât termenul proximal din FedProx, cât și variabilele de control din SCAFFOLD sau regularizatorul din FedDyn. Aplicat în acest context, un astfel de termen de corecție ar trage actualizarea locală către un punct de referință arbitrar, fără ca vreo garanție de convergență din literatură să mai fie valabilă, întrucât toate cele trei presupun, în demonstrațiile lor, runde sincrone cu mai mulți clienți. De aceea, design-ul asincron al CultureQuest are nevoie de altceva - un mecanism care reduce influența actualizărilor venite de la clienți cu un model local prea vechi (FedAsync, menționat în secțiunea anterioară). Tabelul 4 rezumă această comparație.

Tabelul 4: Comparație între FedAvg și algoritmi cu corecție explicită a derivatei (FedProx, SCAFFOLD, FedDyn), și soluția adoptată de CultureQuest.

Algoritm	Sincron?	Mecanism de corecție a derivatei	1 client/rundă?
FedAvg	Da	-	Da
FedProx [9]	Da	Termen proximal (penalizare față de modelul global)	Nu
SCAFFOLD [7]	Da	Variabilă de control per client	Nu
FedDyn [1]	Da	Regularizator dinamic per client	Nu
CultureQuest	Nu	Reducere în funcție de vechime	Da

3.5 Tehnologiile utilizate și justificarea acestora

Alegerea tehnologiilor pentru CultureQuest a fost ghidată de câteva criterii comune: compatibilitate *cross-platform* pentru aplicația mobilă, un ecosistem care se integrează natural cu componenta FL și cu inferența modelului, și, unde a fost posibil, soluții *open-source* care pot fi găzduite local. Secțiunile următoare detaliază aceste alegeri pentru fiecare componentă în parte.

3.5.1 Flutter și Riverpod

Pentru aplicația mobilă, CultureQuest folosește Flutter⁵, cu Riverpod⁶ pentru gestionarea stării. Flutter permite o singură bază de cod Dart pentru Android și iOS, cu *hot reload* care accelerează dezvoltarea. Alternativele posibile sunt fie React Native, cu un *bridge* JavaScript peste componente native - deci un strat de interoperabilitate suplimentar - fie dezvoltare nativă pe cele două platforme, caz în care logica clientului FL s-ar duplica pentru fiecare platformă. Riverpod oferă un model de gestionare a stării declarativ și testabil, potrivit pentru o aplicație în care starea ecranului curent (hartă, profil, status FL) trebuie sincronizată cu procese care rulează în fundal, precum antrenarea locală a modelului.

3.5.2 FastAPI, MongoDB și Redis

Pe partea de *backend*, CultureQuest folosește FastAPI⁷, MongoDB⁸ și Redis⁹. FastAPI este scris în Python, ceea ce permite integrarea directă a serviciilor de agregare FL și a antrenării

⁵Documentația oficială Flutter. <https://flutter.dev> (accesat 19.06.2026).

⁶Documentația oficială Riverpod. <https://riverpod.dev> (accesat 19.06.2026).

⁷Documentația oficială FastAPI. <https://fastapi.tiangolo.com> (accesat 19.06.2026).

⁸Documentația oficială MongoDB. <https://www.mongodb.com/docs/> (accesat 19.06.2026).

⁹Documentația oficială Redis. <https://redis.io/docs/> (accesat 19.06.2026).

MLP cu același ecosistem (PyTorch) folosit la pre-antrenare, fără un strat de interoperabilitate între limbaje, iar suportul nativ pentru operații asincrone se potrivește cu un număr mare de clienți care se conectează intermitent, specific design-ului *cross-device*. MongoDB stochează date variabile legate de profiluri de utilizatori, obiective turistice, misiuni, recenzii - mai flexibil decât o schemă de tip relațional, în timp ce Redis acoperă starea efemeră a procesului FL, pentru care un magazin în memorie e suficient și mai rapid decât o bază de date persistentă.

3.5.3 Motorul de Rutare OSRM

Funcționalitatea de rutare se bazează pe OSRM (*Open Source Routing Machine*)¹⁰, un motor *open-source* care poate fi găzduit local alături de restul serviciilor de *backend*, fără dependența de un API extern plătit. OSRM calculează rute pe rețeaua de drumuri pornind de la date OpenStreetMap și răspunde suficient de rapid pentru a fi folosit interactiv, la fiecare actualizare a poziției utilizatorului în timpul parcurgerii unei rute.

3.5.4 Perceptron multi-strat

Modelul de recomandare al CultureQuest este un perceptron multi-strat (MLP, *Multi-Layer Perceptron*). Este vorba despre o arhitectură simplă, cu un număr mic de straturi complet conectate, pornind de la cele 22 de caracteristici asociate obiectivelor turistice și culturale. Pe dispozitiv, modelul nu este folosit doar pentru antrenarea locală, ci și pentru afișare: de fiecare dată când utilizatorul deschide fișa unui obiectiv turistic, aplicația face o nouă trecere *forward pass* prin model, pentru a calcula scorul de potrivire afișat. Această utilizare repetată, la fiecare interacțiune, cântărește în alegerea unei implementări proprii, cât mai ușoară și fără dependențe externe.

Alternativele cunoscute pentru rularea unui model de tip rețea neuronală pe dispozitiv sunt TFLite¹¹ și PyTorch Mobile¹², ambele *runtime-uri* dedicate, cu propriile formate de model și dependențe native pentru fiecare platformă. Pentru un MLP de dimensiunile celui folosit de CultureQuest, aceste *runtime-uri* ar adăuga o dependență de zeci de *megabytes* doar pentru a executa câteva înmulțiri de matrice, fără avantaje reale, cum ar fi operatori optimizați pentru modele mari sau accelerare hardware. Tabelul 5 rezumă această comparație: implementarea proprie, în Dart, rămâne mai mică, fără dependențe de *runtime* suplimentare, și nativ compatibilă cu Flutter.

¹⁰Documentația oficială OSRM. <https://project-osrm.org> (accesat 19.06.2026).

¹¹Documentația oficială TensorFlow Lite. <https://www.tensorflow.org/lite> (accesat 19.06.2026).

¹²PyTorch Mobile este parțial înlocuit de ExecuTorch; documentația oficială ExecuTorch. <https://docs.pytorch.org/executorch/stable/index.html> (accesat 19.06.2026).

Tabelul 5: Comparație între implementarea MLP proprie și runtime-urile dedicate (TFLite, PyTorch Mobile) pentru inferență pe dispozitiv.

Implementare	Dimensiune model	Dependențe de runtime	Compatibil cu Flutter
MLP propriu (Dart)	Redusă - doar parametrii modelului	Niciuna	Da, cod nativ Dart
TFLite	Model + runtime (zeci de MB)	Bibliotecă nativă per platformă	Da, prin plugin
PyTorch Mobile	Model + runtime (zeci-sute de MB)	Bibliotecă nativă per platformă	Da, prin plugin

4 SOLUȚIA PROPUȘĂ

4.1 Prezentare generală a sistemului

4.2 Arhitectura aplicației mobile

4.3 Arhitectura serviciilor *backend*

4.4 Proiectarea sistemului de învățare federată

4.4.1 Arhitectura modelului

4.4.2 Antrenarea locală

4.4.3 Agregare asincronă și reducerea în funcție de vechime

4.4.4 Limitarea gradientilor

4.4.5 Confidențialitate diferențială

4.4.6 Controlul concurenței

4.5 Fluxul de generare a rutelor

4.6 Proiectarea elementelor de gamificare

4.7 Hartă și navigare

4.8 Arhitectura de confidențialitate

5 DETALII DE IMPLEMENTARE

5.1 Implementarea aplicației mobile

5.1.1 Serviciul client de învățare federată

5.1.2 Adăugarea de zgomot pentru asigurarea confidențialității diferențiale

5.1.3 Bufferul local de interacțiuni și persistența datelor

5.1.4 Stocarea temporară a obiectivelor pentru funcționarea offline

5.1.5 Algoritmul de generare a rutelor

5.1.6 Sistemul de misiuni și tehnologia de *geofencing*

5.2 Implementarea serviciilor *backend*

5.2.1 Serviciul de agregare

5.2.2 Punctele de acces API

5.2.3 Sistemul de recenzii și moderarea conținutului

5.3 Fluxul etapei de pre-antrenare

5.4 *Framework-ul* pentru simularea procesului de învățare federată

5.5 Dificultăți întâmpinate în procesul de implementare

6 EVALUARE

6.1 Corectitudinea funcțională

6.2 Performanța modelului de învățare federată

6.2.1 Rezultatele etapei de pre-antrenare

6.2.2 Rezultatele simulării procesului de învățare federată

6.3 Evaluarea confidențialității și robusteții

6.4 Analiză comparativă și discuții

7 CONCLUZII

7.1 Concluzii generale

7.2 Direcții de cercetare și lucrări viitoare

BIBLIOGRAFIE

- [1] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama. Federated learning based on dynamic regularization. In *International Conference on Learning Representations (ICLR)*, 2021.
- [2] G. Adomavicius and A. Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005.
- [3] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary. Federated learning with personalization layers. *arXiv preprint arXiv:1912.00818*, 2019.
- [4] K. Bonawitz, H. Eichner, N. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konecny, S. Mazzocchi, H. B. McMahan, T. Van Overveldt, D. Petrou, D. Ramage, and J. Roselander. Towards federated learning at scale: System design. In *Proceedings of the 2nd SysML Conference*, 2019.
- [5] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou. Mobile recommender systems in tourism. *Journal of Network and Computer Applications*, 39:319–333, 2014.
- [6] P. Kairouz, H. B. McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2), 2021.
- [7] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh. Scaffold: Stochastic controlled averaging for federated learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [8] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3), 2020.
- [9] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020.
- [10] L. Lyu, H. Yu, and Q. Yang. Threats to federated learning: A survey. *arXiv preprint arXiv:2003.02133*, 2020.
- [11] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.
- [12] C. Xie, S. Koyejo, and I. Gupta. Asynchronous federated optimization. *arXiv preprint arXiv:1903.03934*, 2019.

- [13] Q. Yang, Y. Liu, T. Chen, and Y. Tong. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology*, 10(2):12, 2019.

ANEXE

A EXTRASE DE COD